Exception Handling

Module-2



Dr.M. Sivakumar
Professor & Technical Trainer / CSE
Saveetha School of Engineering, SIMATS
sivakumarm.sse@saveetha.com
+91-9500600868

Exception

- During the execution of a program if any unwanted or unexpected error comes that is called an **exception**.
- It means if anything happens wrong during the execution of a program and disrupts the normal flow of the program's instructions is called an **exception.**

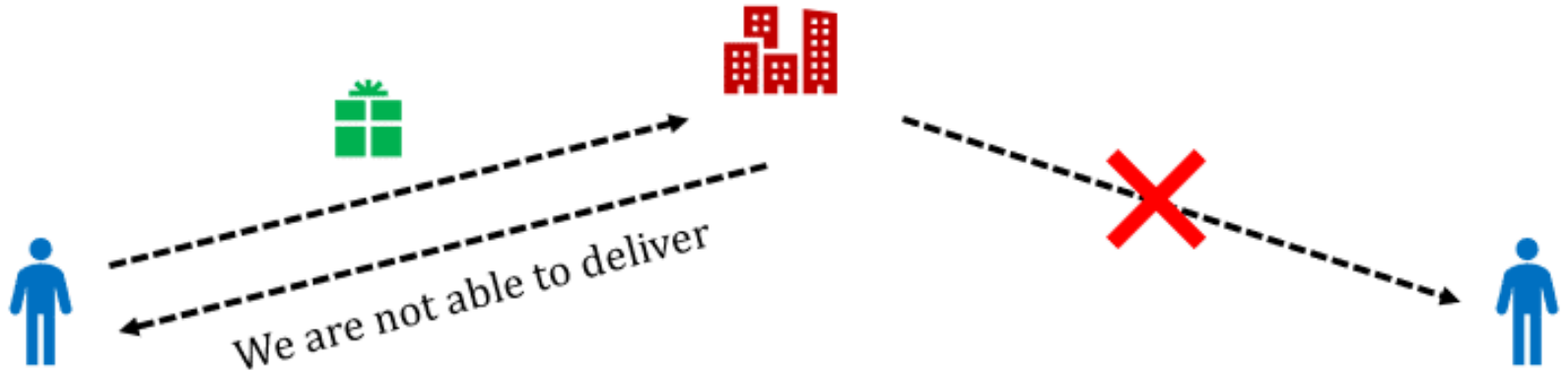
Exceptions may occur for different reasons:

- ✓ If the user has entered the wrong data. Like user is trying to divide any value by 0 e.g. 5/0.
- ✓ If the user tries to open a file with an invalid name.
- ✓ If a user tries to access an array with a negative index value.
- ✓ If a user tries to print a page but the printer doesn't find it.

Exception

Real-time Example

- **Courier company.** If I am sending a gift to my friend but due to some reason the courier company is failed to deliver. That is called **exception** because this is an unexpected thing happened during normal delivery of our product. If a company handling these types of problem and send a notification to me why they were not able to deliver it on time. This handling is called **exceptional handling**.



Exception

Example

- In Java, if any program is disrupted by exception during execution. Then **exception handling** is used to handle exceptions.

```
class FirstExampleForException
{
    public static void main(String args[])
    {
        String str = null;
        System.out.println(str.length());
    }
}
```

Output : *Exception in thread "main" java.lang.NullPointerException at FirstExampleForException.main(FirstExampleForException.java:4)*

Exception Handling

- It is a mechanism to handle exceptions during the execution of the program and maintain the normal flow of execution.
- In other words, we can say it is a way to handle runtime errors. Java provides many keywords to handle the exceptions.

Mechanisms used are:

- ✓ try
- ✓ catch.
- ✓ finally.
- ✓ throw
- ✓ throws

Exception Handling Mechanism

Keyword	Description
try	This keyword is used to specify a block and this block must be followed by either catch or finally. That is, we can't use try block alone.
catch	This keyword must be preceded by a try block to handle the exception and can be followed by a final block later.
finally	This keyword is used to execute the program, whether an exception is handled or not.
throw	This keyword is used to throw an exception.
throws	This keyword is used to declare exceptions.

Example – ‘try’ and ‘catch’

Syntax:

```
try
{
Statements: //generates an exception
}
catch(Exception-type e)
{
Statements; //handling the exception
}
```

Example

```
public class ExceptionDemo
{
    public static void main (String[] args)
    {
    int a=10;
    for(int i=3;i>=0;i--)
    try{
        System.out.println(a/i);
     }
    catch(ArithmeticException e){
        System.out.println(e);
     }
    }
}
```

Output:

```
3
5
10
java.lang.ArithmeticException: / by zero
```

‘finally’

- *A finally block* contains all the crucial statements that must be executed whether an exception occurs or not. i.e important statements are placed here.
- The statements present in this block will always execute, regardless an exception occurs in the try block or not such as closing a connection, stream etc.

```
class SampleFinallyBlock{
 public static void main(String args[]){
  try{
    int data=55/5;
    System.out.println(data);
   }
```

```
catch(NullPointerException e)  {
System.out.println(e);
}
    finally {
            System.out.println("finally block is executed");}
            System.out.println("remaining code");
 } }
```

‘throw’

- It is a keyword that is used to explicitly throw an exception.
- We can use throw where according to our logic an exception should occur.

```
public class ExceptionDemo {
    static void canVote(int age){
        if(age<18)
      try{
        throw new Exception();
      }catch(Exception e){
        System.out.println("you are not an adult!");
      }
        else
          System.out.println("you can vote!");
    }
    public static void main (String[] args) {
        canVote(20);
        canVote(10);
    }
}
```

Output:

```
you can vote!
you are not an adult!
```

‘throws’

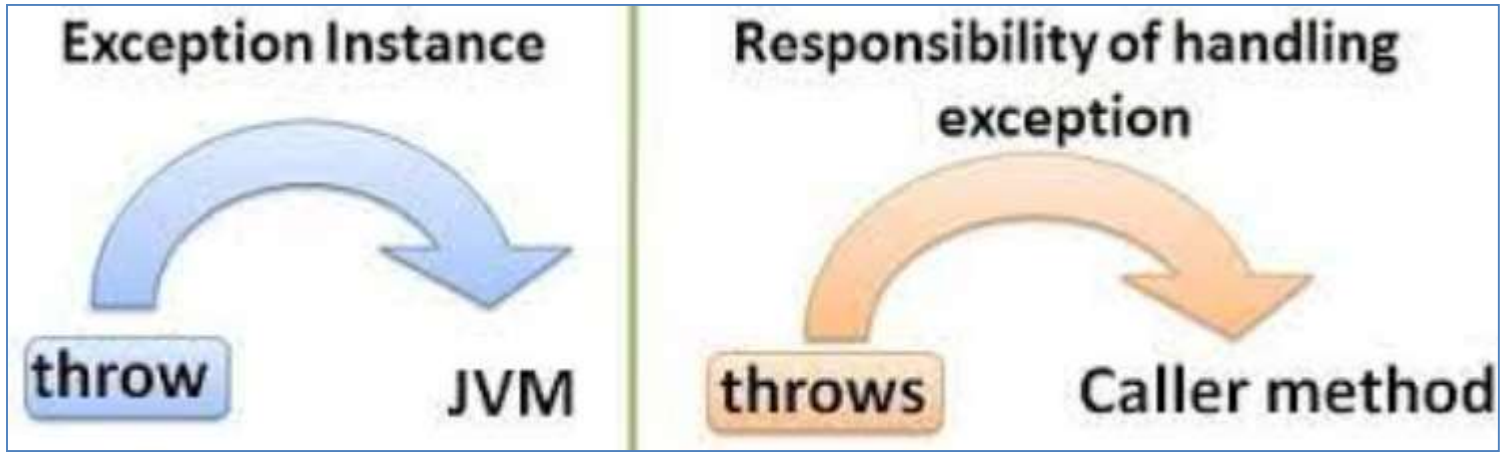
- Throws keyword is used when callee doesn’t want to handle the exception rather it wants to extend this responsibility of handling the exception to the caller of the function.
- Basically says what sort of exception the code can throw and relies on the caller to handle it.
- It is used to handle checked Exceptions as the compiler will not allow code to compile until they are handled.

```
public class ExceptionDemo {
    static void func(int a) throws Exception{
        System.out.println(10/a);
    }
    public static void main (String[] args) {
        try{
            func(10);
            func(0);
        }catch(Exception e){
          System.out.println("can't divide by zero");
        }

    }
}
```

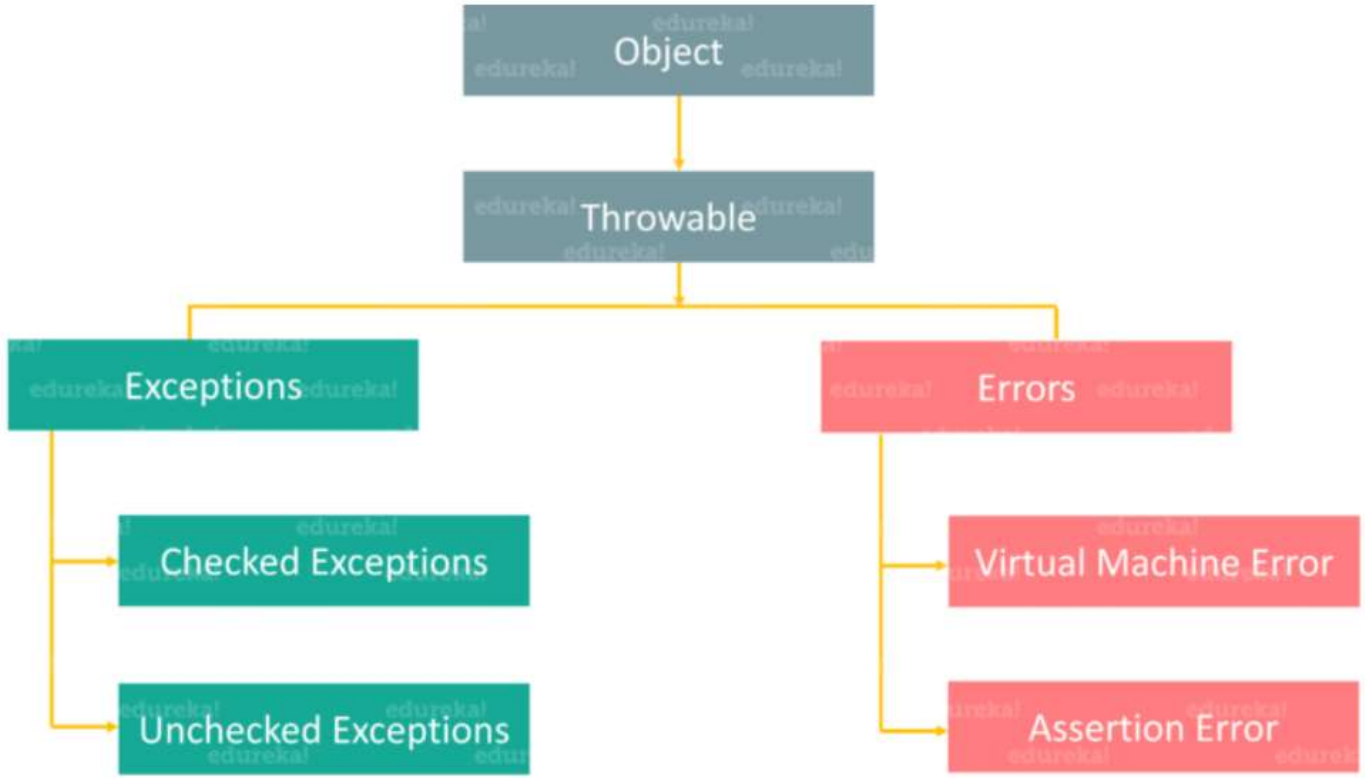
‘throw’ vs ‘throws’

throw	throws
1. Used to explicitly throw an exception	1. Used to declare an exception
2. Checked exceptions cannot be propagated using throw only	2. Checked exceptions can be propagated
3. Followed by an instance	3. Followed by a class
4. Used within a method	4. Used with a method signature
5. Cannot throw multiple exceptions	5. Can declare multiple exceptions



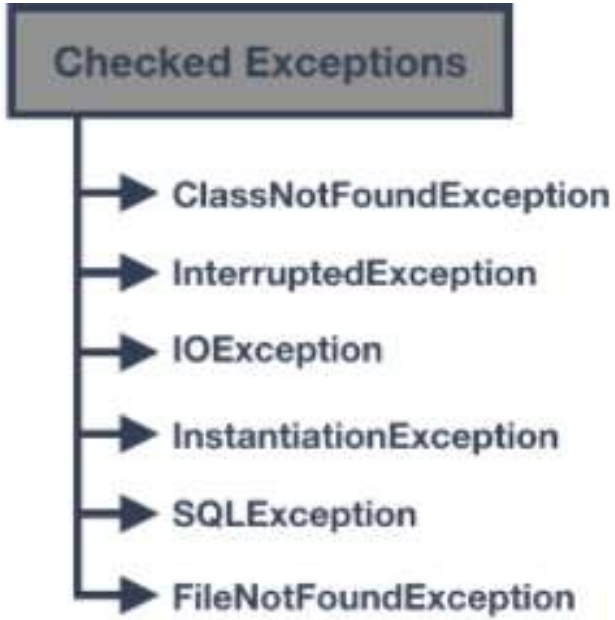
Exception Hierarchy

- All exception and error types are subclasses of class ***Throwable***, which is the base class of hierarchy.
- One branch is headed by ***Error*** which occurs at run-time and other by ***Exception*** that can happen either at compile time or run-time.



1.Checked Exception

- **Checked exceptions in java** are also known as **compile-time** exceptions because these exceptions are checked by the compiler at compile time.
- If any method **throws** a checked exception, then it is programmer responsibility either handle the exception or throw the exception by using **throws** keyword.
- We can't ignore these exceptions at compile time. The JVM checks these exceptions at the time of compilation time.



1.Checked Exception

```
class Main {
  public static void main(String args[]) throws IOException
  {
   FileInputStream fis = null;
   fis = new FileInputStream("C:/myfile.txt");
   int k;
  while(( k = fis.read() ) != -1)  {
      System.out.print((char)k);
   }
   fis.close();
  }
}
```

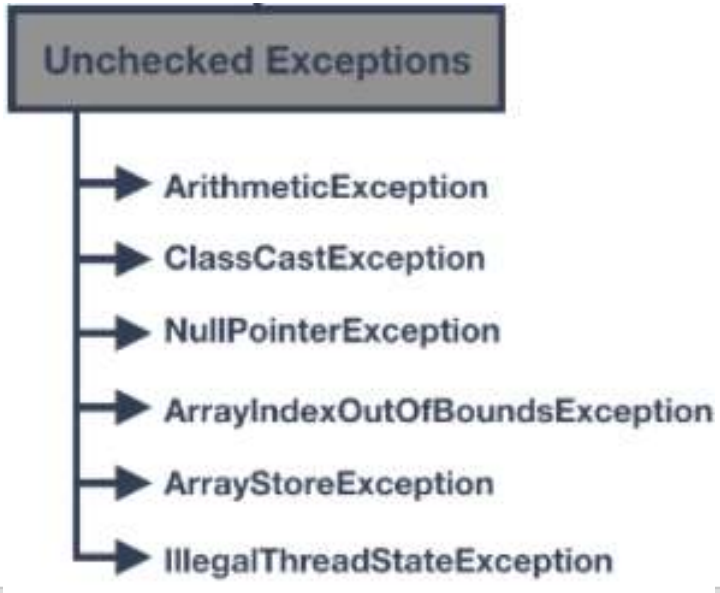
Output

```
Exception in thread "main" java.io.FileNotFoundException: C:/myfile.txt (No
such file or direc
tory)
```

2.Un-Checked Exception

- **Unchecked exception in java** is the exception that is not checked at the compiled time.
- The compiler is not able to recognize these exceptions at compile time.
- An **unchecked exception in java** occurs at the time of execution, so they are also called **Runtime Exceptions**.

  For example **NullPointerException**, **ArithmeticException**, **ArrayIndexOutOfBound**, logic error, etc.



2.Un-Checked Exception

```
class Main {
    public static void main(String args[])
    {
     int num1=10;
     int num2=0;
     //divide both numbers and print the result
     int result=num1/num2;
     System.out.println(result);
    }
}
```

Output

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at Main.main(Main.java:9)
```

Multiple Catch exception

- We can place multiple catch blocks after a try block in Java.
- If there are multiple exception classes in the try block, then Java allows several catch blocks to handle them.

```
try
{
//protected code
}
catch (ExceptionType1 exception1)
{
//Catch block1
}
catch (ExceptionType2 exception2)
{
//Catch block2
}
catch (ExceptionType3 exception3)
{
    //Catch block3
}
```

Multiple Catch exception

```
public class MultipleCatchBlockDemo {
  public static void main(String[] args) {
    try {
      int array[] = new int[5];
      array[5] = 30 / 0;
      System.out.println(a[8]);
    }
    catch(ArithmeticException e) {
      System.out.println("Arithmetic Exception");
    }
    catch(ArrayIndexOutOfBoundsException e) {
      System.out.println("ArrayIndexOutOfBounds");
    }
    catch(Exception e) {
      System.out.println("Parent Exception");
    }
    System.out.println("Rest code");
  }
}
```

Output:

```
Arithmetic Exception
Rest code
```

Custom exception

- In java we can create our own exception class and throw that exception using throw keyword. These exceptions are known as **user-defined** or **custom** exceptions

Key points :

- Naming Convention: All the exception classes provided by the JDK end with Exception. So, your custom exception should follow naming convention.
- Extends Exception class: If you are creating custom exception class then you must extend the Exception class.
- Constructor: Its not mandatory to create any constructor in custom exception class. But its good habit to provide a parameterized constructor in custom exception class.

Custom exception

```
class MyException extends Exception{
 String str1;
 MyException(String str2) {str1=str2;}
  public String toString(){
   return ("MyException Occurred: "+str1);
 }
}
public class MyClass{
public static void main(String args[]){
 try{
      System.out.println("Start of try block");
      throw new MyException("Error Message");
    }
    catch(MyException exp){System.out.println("Catch Block");
    System.out.println(exp);
    }
}
}
```

```
Start of try block
Catch Block
MyException Occurred: Error Message
```

Question

Java program that prompts the user to enter a length in feet and inches and outputs the equivalent length in centimetres. If the user enters a negative number or a non-digit number, throw and handle an appropriate exception and prompt the user to enter another set of numbers


Thank
you!